

6교시: GitHub Actions 1 - 코드, Workflow, Unit/SAST/DAST Gate

Week 3 Day 3 Lesson 6 - Timeout과 Retry 기본

네트워크 지연, 장애 상황에서 요청이 무한히 기다리거나 실패할 수 있습니다. timeout으로 무한 대기를 방지하고, retry로 일시적 장애를 극복합니다.

1. timeout: 무한 대기 방지

timeout 없이 (무한 대기)

클라이언트 → 요청 → 서비스 → 응답 없음...

⌚ 계속 기다림 (Hang)

- 사용자 경험 나쁨
- 연결 자원 고갈
- 시스템 전체 지연 전파

timeout 설정 (예: 3초)

클라이언트 → 요청 → 서비스 → 3초 후 → 실패

✅ 빠른 실패 (Fail Fast)

- 무한 대기 방지
- 자원 회수 및 시스템 보호
- 상위 로직에서 재시도/대체 처리 가능

2. retry: 일시적 장애 극복

클라이언트 → 1차 시도 (요청) → 서비스 (Timeout/오류) → 대기 간격 (Backoff) → 2차 시도 (요청) → 서비스 (Timeout/오류) → 대기 간격 (Backoff) → 3차 시도 (요청) → 서비스 (성공)

최종 결과: 성공 또는 실패 반환

- 실패 시 상위 로직으로 오류 전달

🔄 재시도 횟수 (예: 최대 3회) 📈 Backoff 전략 (지수 백오프 권장) 🌊 Jitter 적용 (동시 재시도 폭주 방지)

3. 중복 처리 위험과 idempotency

⚠️ 중복 처리 위험: 재시도 시 같은 작업이 여러 번 실행될 수 있음

1차 시도 (요청) → 서비스 (실패) → 2차 시도 (요청) → 서비스 (성공) → 3차 시도 (요청) → 서비스 (성공)

결과: 3번 실행

🛡️ 해결: Idempotency (멱등성)
같은 요청을 여러 번 보더라도 한 번만 처리되도록 보장

📋 방법 예시

- 멱등키(Idempotency-Key) 사용
- 고유 요청 ID 저장 후 중복 요청 무시
- UPSERT/조건부 업데이트 사용
- 상태 검사 후 이미 처리된 요청은 무시

📌 영향 예시

- 주문 생성 중복
- 결제 중복
- 메시지 중복 발송
- 데이터 중복 저장

4. 실패 후 복구 전략

1. 감지 (Timeout/오류 발생)
2. 재시도 (Backoff 적용)
3. 대체 처리 (Fallback/대체 경로)
4. 복구 확인 (상태 확인 및 정상화)

5. 실습 체크 포인트

- timeout을 적절히 설정했는가?
- 재시도 횟수와 backoff를 설정했는가?
- 중복 처리 위험을 고려했는가?
- idempotency를 적용했는가?
- 실패 후 복구 경로를 준비했는가?
- 로그와 메트릭을 수집하고 있는가?

💡 TIP
너무 짧은 timeout은 불필요한 실패를 유발, 너무 긴 timeout은 자원 고갈을 유발합니다. 서비스 특성에 맞게 조정하세요!

★ 핵심 요약

- timeout으로 무한 대기 방지
- retry로 일시적 장애 극복 (재시도 횟수 제한)
- backoff와 jitter로 재시도 폭주 방지
- idempotency로 중복 처리 위험 제거
- 실패 후 복구 전략으로 안정성 확보

수업 목표

- GitHub Actions로 빌드할 sample app 구조를 이해한다.
- .gitignore, .dockerignore가 왜 필요한지 확인한다.
- unit test, SAST, DAST가 서로 무엇을 검증하는지 구분한다.
- Docker image를 로컬에서 build/run하여 Actions 전에 검증한다.
- GitHub Actions workflow YAML을 직접 작성하는 기본 절차를 익힌다.

Sample App 구조

```
find week3/day3/labs/dockerhub-app -maxdepth 2 -type f | sort
```

구성:

파일	역할
app.py	/health JSON을 반환하는 작은 HTTP app
Dockerfile	Python alpine 기반 image build
.dockerignore	image build context에서 제외할 파일
README.md	local build/run/pull 절차

.gitignore 확인

```
cat .gitignore
```

확인할 것:

항목	이유
.env	secret 노출 방지
node_modules	dependency output 제외
private/	개인 자료 제외
out_lecture/**/*.md	export 산출물 중 markdown 제외

.dockerignore 확인

```
cat week3/day3/labs/dockerhub-app/.dockerignore
```

확인할 것:

항목	이유
.git/	image에 Git history 포함 방지
.env	secret 포함 방지
cache/build output	image size와 build context 감소

로컬 Docker Build

```
cd /mnt/d/paperclip
week3/day3/labs/quality-gates/unit-test.sh
week3/day3/labs/quality-gates/sast-scan.sh

docker build \
```

```
--build-arg APP_VERSION=0.1.0 \
-t w3d3-dockerhub-app:0.1.0 \
week3/day3/labs/dockerhub-app
```

로컬 실행 검증

```
docker rm -f w3d3-dockerhub-app 2>/dev/null || true
docker run -d --name w3d3-dockerhub-app -p 18088:8080 w3d3-dockerhub-app:0.1.0
curl -s http://localhost:18088/health
docker logs --tail=20 w3d3-dockerhub-app
docker rm -f w3d3-dockerhub-app
```

Unit Test, SAST, DAST

Gate	실행 명령	확인하는 것
unit test	unit-test.sh	app 함수/응답 구조
SAST	sast-scan.sh	위험 코드 패턴, hardcoded secret
DAST	dast-health-check.sh	container 실행 후 HTTP health

한 번에 실행:

```
week3/day3/labs/quality-gates/run-all-local.sh
```

Gate가 늘어나면 느려진다

추가 절차	늘어나는 시간	얻는 것
unit test	짧음	기본 회귀 방지
SAST	짧음~중간	secret/위험 코드 조기 발견
Docker build	중간	artifact 생성 검증
cache restore/save	짧음~중간	반복 build 시간 감소
DAST	짧음~중간	실행 후 health 확인

느려지는 것은 사실이다. 하지만 사람이 수동으로 매번 같은 검증을 기억해서 수행하는 것보다, 자동화된 gate가 더 재현 가능하다.

성공 기준:

Evidence	기준
build	image 생성 성공
curl	status: ok, version: 0.1.0
logs	starting, http_access

Workflow 파일 위치

GitHub Actions workflow는 repository 안의 정해진 경로에 둔다.

```
.github/workflows/dockerhub-publish.yml
```

파일 이름은 자유롭게 정할 수 있지만, 확장자는 보통 .yml 또는 .yaml을 쓴다. GitHub는 이 경로에 있는 workflow 파일을 읽고 Actions 탭에 표시한다.

Workflow 작성 순서

처음부터 Docker Hub push까지 한 번에 쓰려고 하면 헛갈린다. 다음 순서로 눌러간다.

순서	작성할 것	확인할 것
1	name	Actions 화면에 보이는 workflow 이름
2	on	언제 실행할지
3	jobs	어떤 작업 묶음을 실행할지
4	runs-on	어떤 runner에서 실행할지
5	steps	실제 실행 단계
6	uses	이미 만들어진 action 사용
7	run	shell 명령 직접 실행
8	env	반복해서 쓰는 값 정의
9	secrets	token/password 같은 민감 값 참조

최소 Workflow

가장 작은 workflow부터 확인한다.

```
name: w3d3-first-action

on:
  workflow_dispatch:
```

```

jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Print runner info
        run: |
          pwd
          uname -a
          echo "hello github actions"

```

이 workflow는 Docker build를 하지 않는다. 목적은 on, jobs, runs-on, steps, run의 위치를 눈에 익히는 것이다.

Workflow 핵심 문법

문법	의미	예시
name	workflow 이름	w3d3-dockerhub-publish
on.workflow_dispatch	수동 실행 버튼 제공	Actions UI에서 Run workflow
on.push.tags	tag push 때 실행	v*.*.*
jobs.<job_id>	job 식별자	build-and-push
runs-on	runner image	ubuntu-latest
steps[].name	log에 보이는 step 이름	Run unit test
steps[].uses	외부 action 호출	actions/checkout@v4
steps[].run	shell 명령 실행	docker build ...
env	환경 변수	IMAGE_NAME
\${{ secrets.NAME }}	GitHub Secret 참조	\${{ secrets.DOCKERHUB_TOKEN }}
\${{ steps.ID.outputs.KEY }}	이전 step 출력 참조	\${{ steps.meta.outputs.version }}

Runner Computing 이해

GitHub Actions에서 실제 명령을 실행하는 컴퓨팅 자원을 runner라고 부른다.

구분	설명
GitHub-hosted runner	GitHub가 제공하는 실행 환경
self-hosted runner	회사나 개인이 직접 운영하는 실행 환경
runs-on	job이 어떤 runner에서 실행될지 정하는 필드
workspace	runner 안에서 repository가 checkout되는 작업 디렉터리

GitHub-hosted runner는 수업에서 가장 편하게 쓸 수 있다. 하지만 실행할 때마다 깨끗한 환경에서 시작한다고 생각해야 한다. 그래서 로컬 PC처럼 Docker layer cache, dependency cache, build output이 항상 남아 있다고 기대하면 안 된다.

느려지는 지점	이유
checkout	runner가 매번 repository를 새로 가져온다
dependency install	이전 실행에서 설치한 패키지가 기본적으로 남아 있지 않다
Docker build	base image pull, layer rebuild, cache miss가 발생한다
Docker Hub push	image size와 network 상태에 영향을 받는다

GHA Cache

반복 build가 느리면 cache를 명시적으로 사용한다. Docker BuildKit은 GitHub Actions cache backend인 type=gha를 사용할 수 있다.

```
- name: Build local image for DAST
  uses: docker/build-push-action@v6
  with:
    context: ${{ env.APP_DIR }}
    load: true
    tags: ${{ env.IMAGE_NAME }}:${{ steps.meta.outputs.version }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

의미:

설정	의미
cache-from: type=gha	이전 Actions cache에서 build layer를 읽는다
cache-to: type=gha,mode=max	다음 실행을 위해 layer cache를 저장한다
load: true	build한 image를 runner의 local Docker engine에 적재한다
mode=max	중간 layer까지 더 적극적으로 저장한다

cache는 공짜 마법이 아니다. cache를 복원하고 저장하는 시간도 들고, cache key나 build context가 자주 바뀌면 효과가 작다. 그래도 Docker build가 반복되는 수업/팀에서는 cold build와 warm build 시간을 비교해볼 가치가 있다.

Self-Hosted Runner

self-hosted runner는 회사 내부 서버, 클라우드 VM, 온프레미스 장비에 runner를 직접 설치해 사용하는 방식이다.

장점	단점
Docker cache와 dependency cache를 더 오래 유지할 수 있음	runner OS/패치/보안 관리 책임
사내망, private registry, 내부 DB 접근 가능	secret 노출과 workspace 오염 위험 관리 필요
큰 CPU/RAM/Disk 장비 선택 가능	장애 대응과 capacity 관리 필요
특수 도구가 설치된 환경 유지 가능	여러 repo가 공유하면 권한 분리가 어려워질 수 있음

수업에서는 GitHub-hosted runner를 기본으로 사용한다. self-hosted runner는 속도가 필요하거나 내부망 접근이 필요한 회사 환경에서 검토한다.

작성 시 주의사항

주의사항	이유	
들여쓰기 2칸 유지	YAML은 들여쓰기가 문법이다	
`run:`	`아래 명령은 한 단계 더 들여쓰기	multi-line shell block
secret 값을 echo하지 않기	log 노출 위험	
step을 너무 크게 합치지 않기	실패 위치와 실행 시간 분석이 어려움	
local에서 먼저 같은 명령을 실행하기	runner에서 실패하기 전에 문제 확인	

Docker Hub Push Workflow 읽기

```
cat week3/day3/labs/github-actions/dockerhub-publish.yml
```

핵심:

YAML	의미
workflow_dispatch	수동 실행 가능
push.tags	tag push 시 실행
runs-on	GitHub-hosted runner
checkout	repo 코드 가져오기
build-push-action	Docker build/push
cache-from: type=gha	GitHub Actions cache에서 layer 복원
cache-to: type=gha	다음 실행을 위해 layer 저장
unit test step	push 전 코드 검증
SAST step	push 전 보안/secret scan
DAST step	image 실행 후 health 검증

Docker Hub Workflow 작성 흐름

최종 workflow는 다음 흐름으로 작성한다.

```

checkout
-> version metadata 만들기
-> unit test
-> SAST/secret scan
-> Docker build with gha cache
-> DAST health check
-> Docker Hub login
-> Docker Hub push
-> pull/run command 출력
  
```

각 step을 나눠야 Actions 화면에서 어느 단계가 실패했는지, 어느 단계가 오래 걸렸는지 바로 확인할 수 있다.

핵심 포인트

Actions에서 바로 push하기 전에 local gate로 같은 절차를 먼저 검증해야 한다.

```
unit test -> SAST -> docker build -> DAST -> push
```


Evidence Note

```
# W3D3S6 Actions Local Build
- .gitignore checked:
- .dockerignore checked:
- local image:
- curl result:
- unit test:
- SAST:
- DAST:
- workflow file:
```